

Advanced Database Systems

Chapter 1 Query Processing and Optimization

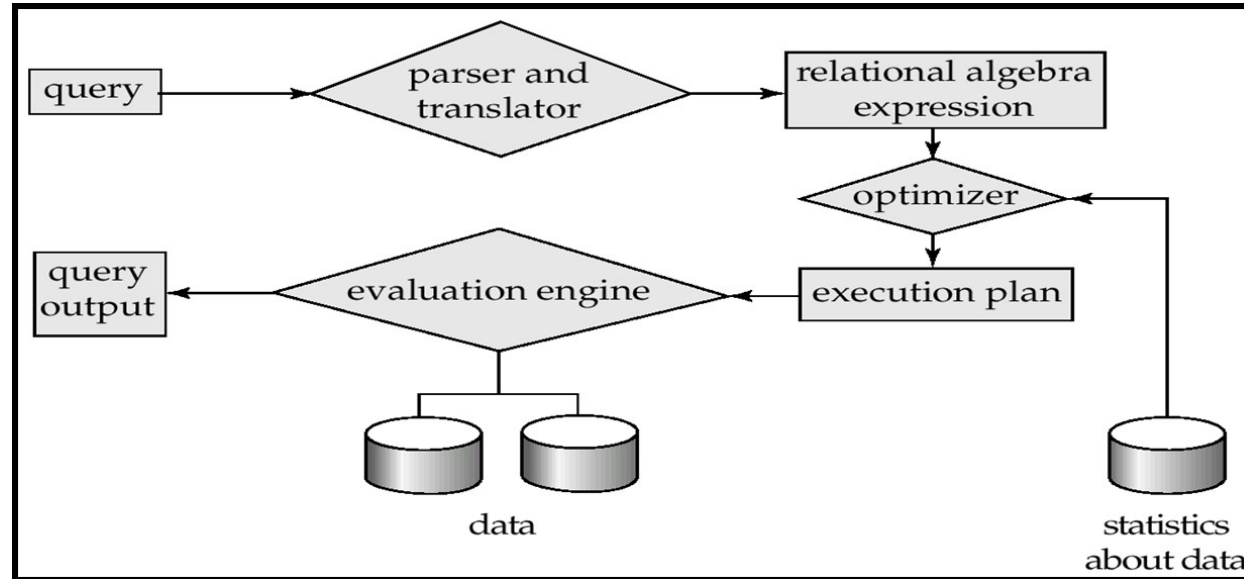


Sirage Z.
Department of Computer Science
Debre Berhan University
2022

Query Processing

- **Query** is a request for data or information from a database table.
- **QP** Is find information in one or more databases and deliver it to the user quickly and efficiently.
- Databases containing more complex and diverse types of data demand new **query processing and optimization techniques**.
- Query Processing can be divided into four main phases:
 - Decomposition:
 - Optimization
 - Code generation, and
 - Execution

Query Processing Steps



➤ **Parser and Translator**:-check correct query *syntax* and *Schema elements*.

✓ *Eg. SELECT empName from employee having name="abc"*

> Having is invalid(where is correct)→syntax error

✓ Assume emp.gmail is not available in the table;

> Select emp.gmail from employee.→schema element error

➤ **Optimizer:**

- ✓ Find best plan to relational algebra
- ✓ Finds all equivalent relational algebra
- ✓ Find the expression with least cost

➤ **Statistics about data:** is the size of relation

- ✓ Number of blocks been used to store the relation
- ✓ Number of records in the relation.
- ✓ Number of index

➤ **Query Evaluation engine:**

- ✓ Evaluates the above plan and gets the results.
- ✓ Is directly accessing data warehouse.

Phases of Query processing

- Processing can be divided into : **Decomposition, Optimization , and Execution ,Code generation**
- **Query Decomposition**
 - ✓ is the process of transforming a high level query into a relational algebra query, and to check that the query is **syntactically** and **semantically** correct.
 - ✓ Consists of parsing and validation
- Typical stages in query decomposition are:
 - ✓ **1) Analysis:** **lexical** and syntactical analysis of the query (correctness).
 - > Query tree will be built for the query containing leaf node for base relations, one or many non-leaf nodes for relations produced by relational algebra operations and root node for the result of the query.
 - > Sequence of operation is from the leaves to the root.

...CON'T

- ✓ **2) Normalization:** convert the query into a normalized form.
 - > The predicate WHERE will be converted to **Conjunctive** ($\wedge$) or **Disjunctive** ($\vee$) Normal form.
- ✓ **3) Semantic Analysis:** to reject normalized queries that are not correctly formulated or contradictory.
 - > Incorrect if components do not contribute to generate result.
 - > Contradictory if the predicate cannot be satisfied by any tuple.
 - > **Algorithms:** *relation connection graph* and *normalized attribute connection graph*.
- ✓ **4) Simplification:** to detect redundant qualifications, eliminate common sub-expressions, and transform the query to a semantically equivalent but more easily and effectively computed form.
- ✓ **5) Query Restructuring** More than one translation is possible Use transformation rules.
 - > Re arranging nodes so that the most restrictive condition will be executed first

...CON'T

- Consider the following query from two relations; staff and branch
 - ✓ Staff (StaffNo, name, position, salary, sex, BranchNo)
 - ✓ Branch (BranchNo, name, city)
- **Eg.** Find all managers who manage a branch at Addis.
- We can write this query in a SQL as follows

*Select S.**

from staff S, branch b

*where branchNo=S.branchNo **AND***

*(S.position='Manager' **AND** b.city='Addis')*

...CON'T

- *Assume*

- i. One record is accessed at a time, n staff, m branches, x non-manager, and y non-Addis branches for some integers n, m, x, y .*
 - ii. intermediate results are stored on disk*
 - iii. ignore about the final result(write) because it is the same for all the expressions*

- Then, this high level SQL can be transformed in the following three low level equivalent relational algebra expressions.

- $\sigma(\text{position}=\text{'manager'}) \wedge (\text{City}=\text{'Addis'}) \wedge$
 $(\text{staff.branchNo}=\text{branch.branchNo}, (\text{Staff X Branch}))$

- *Analysis:*

- i. read each tuple from the two relations $\rightarrow n+m$ reads*
 - ii. create a table of the Cartesian product $\rightarrow n \times m$ writes*
 - iii. test each tuple of step 2 $\rightarrow n \times m$ read*

- Total No. of Disk access: $\rightarrow 2(n \times m) + n + m$ or

...CON'T

OR

- $\sigma(\text{position}='manager') \wedge (\text{City}='Addis')(\text{Staff} \bowtie \text{staff.branchNo}=\text{branch.branchNo} \bowtie \text{Branch})$
- Analysis:
 - i. read each tuple from the two relation $\rightarrow n+m$ reads
 - ii. create a table of the Join $\rightarrow n$ writes
 - iii. test each tuple of step 2 $\rightarrow n$ read
- Total No. of Disk access: $\rightarrow 3(n) + m$ Or
- $(\sigma_{\text{position}='manager'}(\text{Staff})) \text{ staff.branchNo}=\text{branch.branchNo} (\sigma_{\text{City}='Addis'}(\text{Branch}))$
- Analysis:
 - i. Test each tuple from the two relations $\rightarrow n+m$ reads
 - ii. create a “manager_Staff” and “addis_Branch” relations $\rightarrow (n-x) + (m-y)$ writes
 - iii. create a join of the two relations at step 2 $\rightarrow (n-x) + (m-y)$ reads
- Total No. of Disk access: $\rightarrow 2(n-x) + 2(m-y) + n + m$
- Which of the expressions given above do you think is best (Optimal)?

...CON'T

Assume:

- 1000 tuples in ***Staff***.
- 50 Managers
- 50 tuples in ***Branch***.
- 5 London branches
- No indexes or sort keys
- All temporary results are written back to disk (memory is small)
- Tuples are accessed one at a time (not in blocks)

...CON'T

➤ Query 1 (Bad)

- $\sigma_{(\text{position}='Manager') \wedge (\text{city}='London') \wedge (\text{Staff.branchNo}=\text{Branch.branchNo})} (\text{Staff} \times \text{Branch})$
- ✓ Requires (1000+50) disk accesses to read from Staff and Branch relations
 - ✓ Creates temporary relation of Cartesian Product (1000*50) tuples
 - ✓ Requires (1000*50) disk access to read in temporary relation and test predicate

**Total Work = (1000+50) + 2*(1000*50) =
101,050 I/O operations**

➤ Query 2 (Better)

- $\sigma_{(\text{position}='Manager') \wedge (\text{city}='London')} (\text{Staff} \bowtie_{\text{Staff.branchNo} = \text{Branch.branchNo}} \text{Branch})$
- Again requires (1000+50) disk accesses to read from **Staff** and **Branch**
 - Joins **Staff** and **Branch** on branchNo with 1000 tuples
(1 employee : 1 branch)
 - Requires (1000) disk access to read in joined relation and check predicate

**Total Work = (1000+50) + 2*(1000) =
3050 I/O operations
33% Improvement over Query 1**

...CON'T

➤ Query 3 (Best)

$[\sigma_{(\text{position}=\text{'Manager'})} (\text{Staff})] \triangleright \triangleleft [\sigma_{(\text{city}=\text{'London'})} (\text{Branch})]$
 $\text{Staff.branchNo} = \text{Branch.branchNo}$

- ✓ Read *Staff* relation to determine 'Managers' (1000 reads)
 - > Create 50 tuple relation (50 writes)
- ✓ Read *Branch* relation to determine 'London' branches (50 reads)
 - > Create 5 tuple relation (5 writes)
- ✓ Join reduced relations and check predicate (50 + 5 reads)

**Total Work = 1000 + 2*(50) + 5 + (50 + 5) =
1160 I/O operations**

8700% Improvement over Query 1

Query Optimization

- What is wrong with the ordinary query?
- Everyone wants the performance of their database to be optimal.
- In particular, there is often a requirement for a specific query or object that is query based, to run faster.
- Problem of query optimization is to find the sequence of steps that produces the answer to user request in the most efficient manner, given the database structure.
- The performance of a query is affected by the tables or queries that underlies the query and by the complexity of the query.
- When data/workload characteristics change
 - ✓ The best navigation strategy changes
 - ✓ The best way of organizing the data changes

...CON'T

- Query optimizers are one of the main means by which modern database systems achieve their performance advantages.
- Given a request for data manipulation or retrieval, **an optimizer will choose an optimal plan for evaluating the request from among the many alternative strategies.**
 - ✓ i.e. there are many ways (access paths) for accessing desired file/record.
- The optimizer tries to select the most efficient (cheapest) access path for accessing the data.
- DBMS is responsible to pick the best execution strategy based various considerations.
- Query optimizers were already among the largest and most complex modules of database systems.

...CON'T

- **Most efficient processing:** Least amount of I/O and CPU resources.
- **Selection of the best method:** In a non-procedural language the system does the optimization at the time of execution.
 - ✓ For optimizing the execution of a query the programmer must know:
 - > File organization
 - > Record access mechanism and primary or secondary key.
 - > Data location on disk.
 - > Data access limitations.
- To write correct code, application programmers need to know how data is organized physically (e.g., which indexes exist)
- To write efficient code, application programmers also need to worry about data/workload characteristics
- One has to cope with change! (Real time changes hence, preferable to give the responsibility of optimization to the DBMS).

...CON'T

- **Example:** Consider relations $r(AB)$ and $s(CD)$. We require $r \times s$.
- **Method 1**
 - a) Load next record of r in RAM.
 - b) Load all records of s , one at a time and concatenate with r .
 - c) All records of r concatenated?
 - > NO: goto a.
 - > YES: exit (the result in RAM or on disk).
- **Performance:** Too many accesses.
- **Method 2: Improvement**
 - a) Load as many blocks of r as possible leaving room for one block of s .
 - b) Run through the s file completely one block at a time.
- **Performance:** Reduces the number of times s blocks are loaded by a factor of equal to the number of r records than can fit in main memory.
- Considerations during query Optimization:
 - Narrow down intermediate result sets quickly. SELECT before JOIN
 - Use access structures (indexes).

Approaches to Query Optimization

➤ **Heuristics Approach**

- ✓ Uses the knowledge of the characteristics of the relational algebra operations and the relationship between the operators to optimize the query.
- ✓ Thus the heuristic approach of optimization will make use of:
 - **Properties of individual operators**
 - **Association between operators**
 - **Query Tree**: a graphical representation of the operators, relations, attributes and predicates and processing sequence during query processing.

...CON'T

- Query tree is composed of three main parts:
 - > **The Leafs:** the base relations used for processing the query/ extracting the required information
 - > **The Root:** the final result/relation as an output based on the operation on the relations used for query processing
 - > **Nodes:** intermediate results or relations before reaching the final result.
- Sequence of execution of operation in a query tree will start from the leaves and continues to the intermediate nodes and ends at the root.
- The properties of each operations and the association between operators is analyzed using set of rules called **TRANSFORMATION RULES**.
- Use of the transformation rules will transform the query to relatively good execution strategy.

Transformation Rules for Relational Algebra

- 1. Cascade of SELECTION: conjunctive SELECTION Operations can cascade into individual Selection Operations and Vice Versa

$$\sigma (c1 \wedge c2 \wedge c3) (R) = \sigma c1(\sigma c2(\sigma c3(R))) \text{ where } c_i \text{ is a predicate}$$

- 2. Commutativity of SELECTION operations

$$\sigma c1(\sigma c2(R)) = \sigma c2(\sigma c1(R)) \text{ where } c_i \text{ is a predicate}$$

- 3. Cascade of PROJECTION: in the sequence of PROJECTION Operations, only the last in the sequence is required

$$\pi L1 \pi L2 \pi L3 \pi L4(R) = \pi L1(R)$$

- 4. Commutativity of SELECTION with PROJECTION and Vice Versa

- If the predicate $c1$ involves only the attributes in the projection list ($L1$), then the selection and projection operations commute

$$\pi \langle a1, a2..an \rangle (\sigma c1(R)) = \sigma c1(\pi \langle a1, a2,,,,,an \rangle (R))$$

$$\text{Where } c1 \in \{a1, a2...an\}$$

...CON'T

○ 5. Commutativity of THETA JOIN/Cartesian Product

$R \bowtie S$ is equivalent to $S \bowtie R$

- Also holds for Equi-Join and Natural-Join

$$(R \bowtie_{c1} S) = (S \bowtie_{c1} R)$$

○ 6. Commutativity of SELECTION with THETA JOIN

- a. If the predicate $c1$ involves only attributes of one of the relations (R) being joined, then the Selection and Join operations commute

$$\sigma_{c1}(R \bowtie_c S) = (\sigma_{c1}(R)) \bowtie_c S$$

- b. If the predicate is in the form $c1 \wedge c2$ and $c1$ involves only attributes of R and $c2$ involves only attributes of S , then the Selection and Theta Join operations commute

$$\sigma_c(R \bowtie S) \equiv (\sigma_{c_1}(R)) \bowtie (\sigma_{c_2}(S))$$

...CON'T

- **7. Commutativity of PROJECTION and THETA JOIN**
- If the projection list is of the form **L1, L2**, where **L1** involves only attributes of R and **L2** involves only attributes of S being joined and the predicate **c** involves only attributes in the projection list, then the PROJECTION and JOIN operations commute as:

$$\pi_{L1,L2} (R \bowtie_c S) = (\pi_{L1} (R)) \bowtie_c (\pi_{L2} S)$$

- However if the join condition **c** contains additional attributes not in projection list $L=L_1 \cup L_2$, say $M=M_1 \cup M_2$ where M_1 is from R and M_2 is from S then the PROJECTION and JOIN operations commute as follows:

$$\pi_{L1,L2} (R \bowtie_c S) = \pi_{L1,L2} ((\pi_{L1,M1} R) \bowtie_c (\pi_{L2,M2} S))$$

- **8. Commutativity of the Set Operations: UNION and INTERSECTION but not SET DIFFERENCE:**

$$R \cap S = S \cap R \text{ and } R \cup S = S \cup R$$

...CON'T

- **9. Associativity of the THETA JOIN, CARTESIAN PRODUCT, UNION and INTERSECTION.**

$$(R \theta S) \theta T = R \theta (S \theta T) \text{ where } \theta \text{ is one of the operations}$$

- **10. Commuting SELECTION with SET OPERATIONS**

$$\sigma_c (R \theta S) = (\sigma_c(R) \theta \sigma_c(S)) \text{ where } \theta \text{ is one of the operations}$$

- **11. Commuting PROJECTION with UNION**

$$\pi_{L1} (S \cup R) = \pi_{L1} (S) \cup \pi_{L1} (R)$$

...CON'T

- Heuristic Approach will be implemented by using the above transformation rules in the following sequence or steps.
- Sequence for Applying Transformation Rules
- 1. Use
 - ✓ Rule-1 → Cascade SELECTION
- 2. Use
 - ✓ Rule-2: Commutativity of SELECTION
 - ✓ Rule-4: Commuting SELECTION with PROJECTION
 - ✓ Rule-6: Commuting SELECTION with JOIN and CARTESIAN
 - ✓ Rule-10: commuting SELECTION with SET OPERATIONS

...CON'T

- 3. Use
 - ✓ Rule-9: Associativity of Binary Operations (JOIN, CARTESIAN, UNION and INTERSECTION). Rearrange nodes by making the most restrictive operations to be performed first (moving it as far down the tree as possible)
- 4. Perform Cartesian Operations with the subsequent Selection Operation
- 5. Use
 - ✓ Rule-3: Cascade of PROJECTION
 - ✓ Rule-4: Commuting PROJECTION with SELECTION
 - ✓ Rule-7: Commuting PROJECTION with JOIN and CARTESIAN
 - ✓ Rule-11: commuting PROJECTION with UNION

Main Heuristic

- The main heuristic is to first apply operations that reduce the size (the cardinality and/or the degree) of the intermediate relation.
- That is:
 - ✓ **a.** Perform SELECTION as early as possible: that will reduce the cardinality (number of tuples) of the relation.
 - ✓ **b.** Perform PROJECTION as early as possible: that will reduce the degree (number of attributes) of the relation.
 - Both **a** and **b** will be accomplished by placing the SELECT and PROJECT operations as far down the tree as possible.
 - ✓ **c.** SELECT and JOIN operations with most restrictive conditions resulting with smallest absolute size should be executed before other similar operations.
 - ✓ This is achieved by reordering the nodes with JOIN.

...CON'T

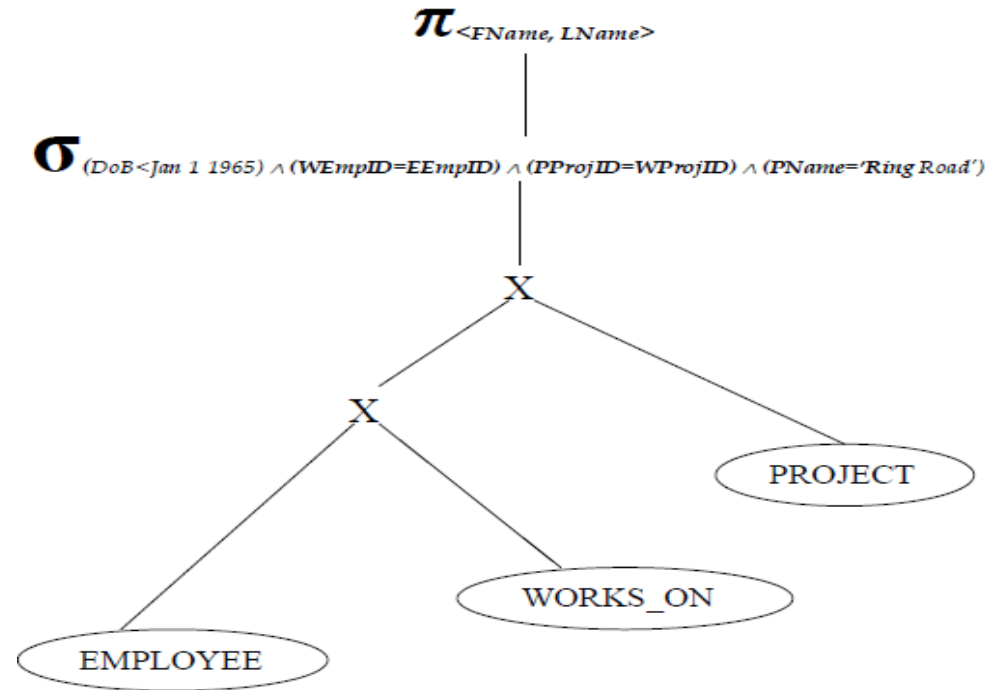
- **Example:** consider the following schemas and the query, where the EMPLOYEE and the PROJECT relations are related by the WORKS_ON relation.
- EMPLOYEE (EEmpID, FName, LName, Salary, Dept, Sex, DoB)
- PROJECT (PProjID, PName, PLocation, PFund, PManagerID)
- WORKS_ON (WEmpID, WProjID)
- WEmpID (refers to employee identification) and PProjID (refers to project identification) are foreign keys to WORKS_ON relation from EMPLOYEE and PROJECT relations respectively.

...CON'T

- **Query:** The manager of the company working on road construction would like to view employees name born before January 1 1965 who are working on the project named Ring Road.
- Relational Algebra representation of the query will be:
- $\pi\langle FName, LName \rangle (\sigma\langle DoB < Jan\ 1\ 1965 \wedge WEmpID = EEmpID \wedge PProjID = WProjID \wedge PName = 'Ring\ Road' \rangle (EMPLOYEE \bowtie WORKS_ON \bowtie PROJECT))$

...CON'T

- The SQL equivalence for the above query will be:
- **SELECT FName, LName FROM EMPLOYEE, WORKS_ON, PROJECT WHERE *DoB* < Jan 1 1965 $\wedge$ *EEmpID* = *WEmpID* $\wedge$ *WProjID* = *PProjID* $\wedge$ *PName* = "Ring Road"**
- The initial query tree will be:



...CON'T

- By applying the first step (cascading the selection) we will come up with the following structure.

$\sigma(\text{DoB} < \text{Jan } 1 \text{ } 1965)$

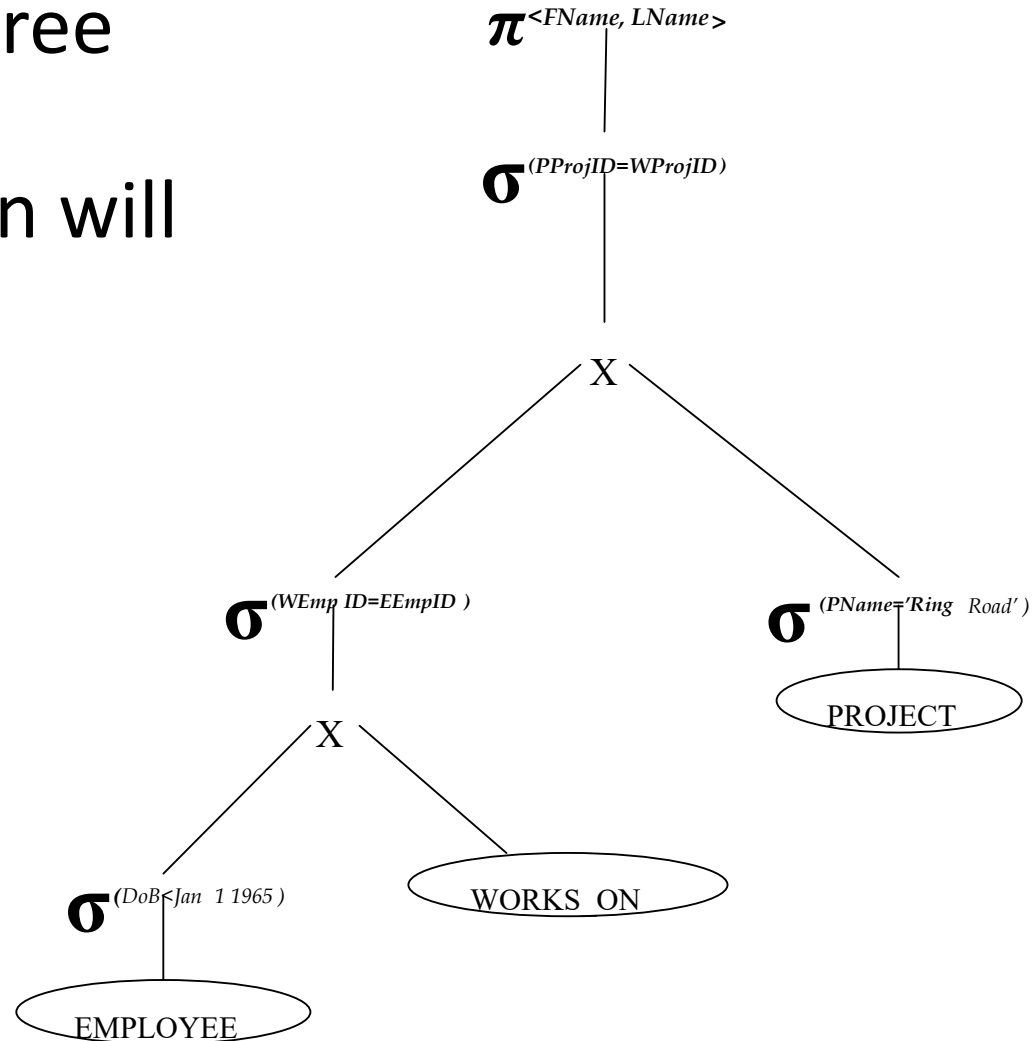
$(\sigma(\text{WEmpID} = \text{EEmpID})(\sigma(\text{PProjID} = \text{WProjID})(\sigma(\text{PName} = \text{'Ring Road'}) (\text{EMPLOYEE } X \text{ WORKS_ON } X \text{ PROJECT})))$

- By applying the second step it can be seen that some conditions have attribute that belong to a single relation (DoB belongs to EMPLOYEE and PName belongs to PROJECT) thus the selection operation can be commuted with Cartesian Operation.
- Then, since the condition **WEmpID=EEmpID** base the employee and WORKS_ON relation the selection with this condition can be cascaded.

$(\sigma(\text{PProjID} = \text{WProjID}) (\sigma(\text{PName} = \text{'Ring Road'}) \text{ PROJECT}) X (\sigma(\text{WEmpID} = \text{EEmpID})$
 $(\text{WORKS_ON } X (\sigma(\text{DoB} < \text{Jan } 1 \text{ } 1965) \text{ EMPLOYEE}))))$

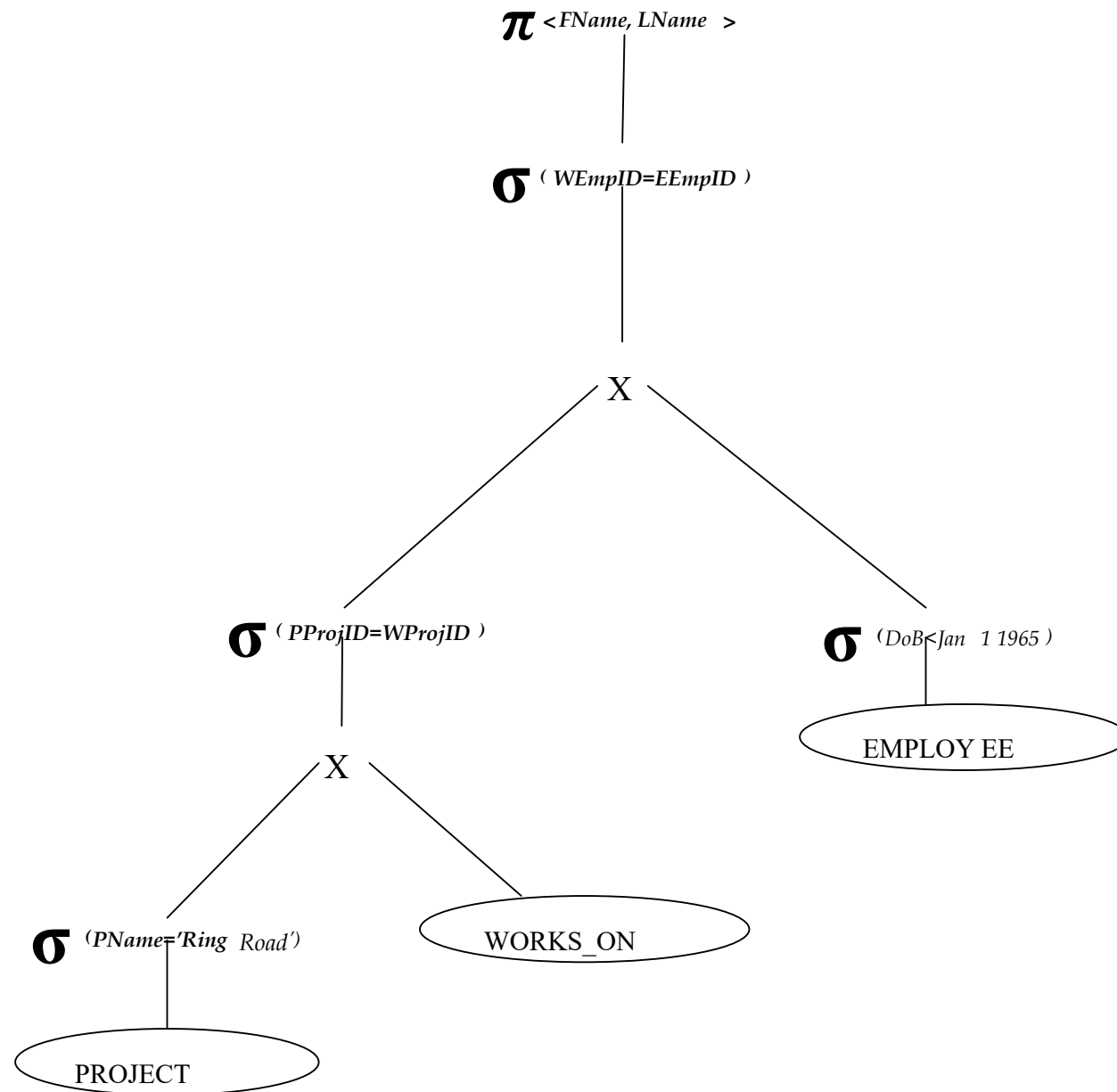
...CON'T

- The query tree after this modification will be:

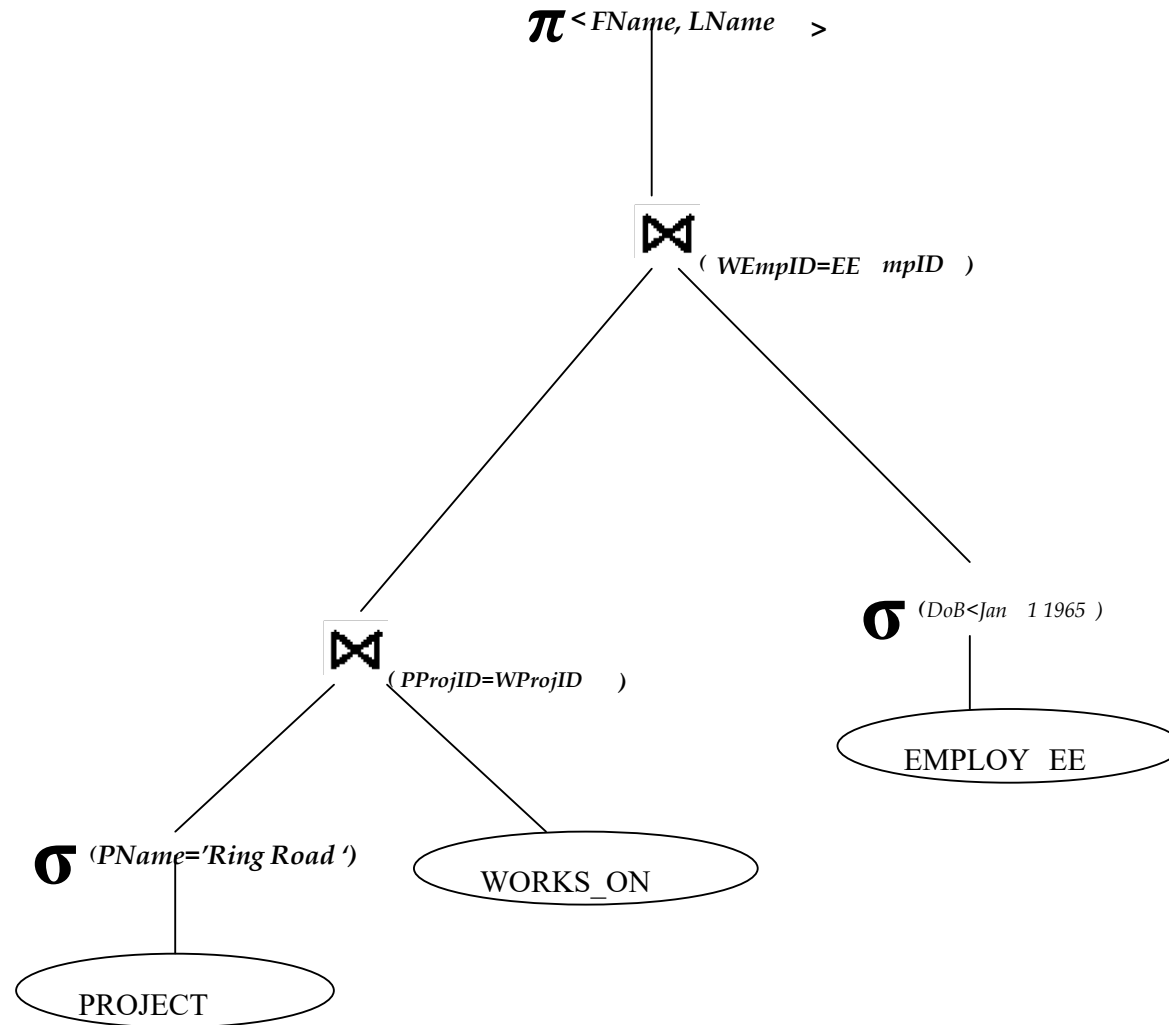


...CON'T

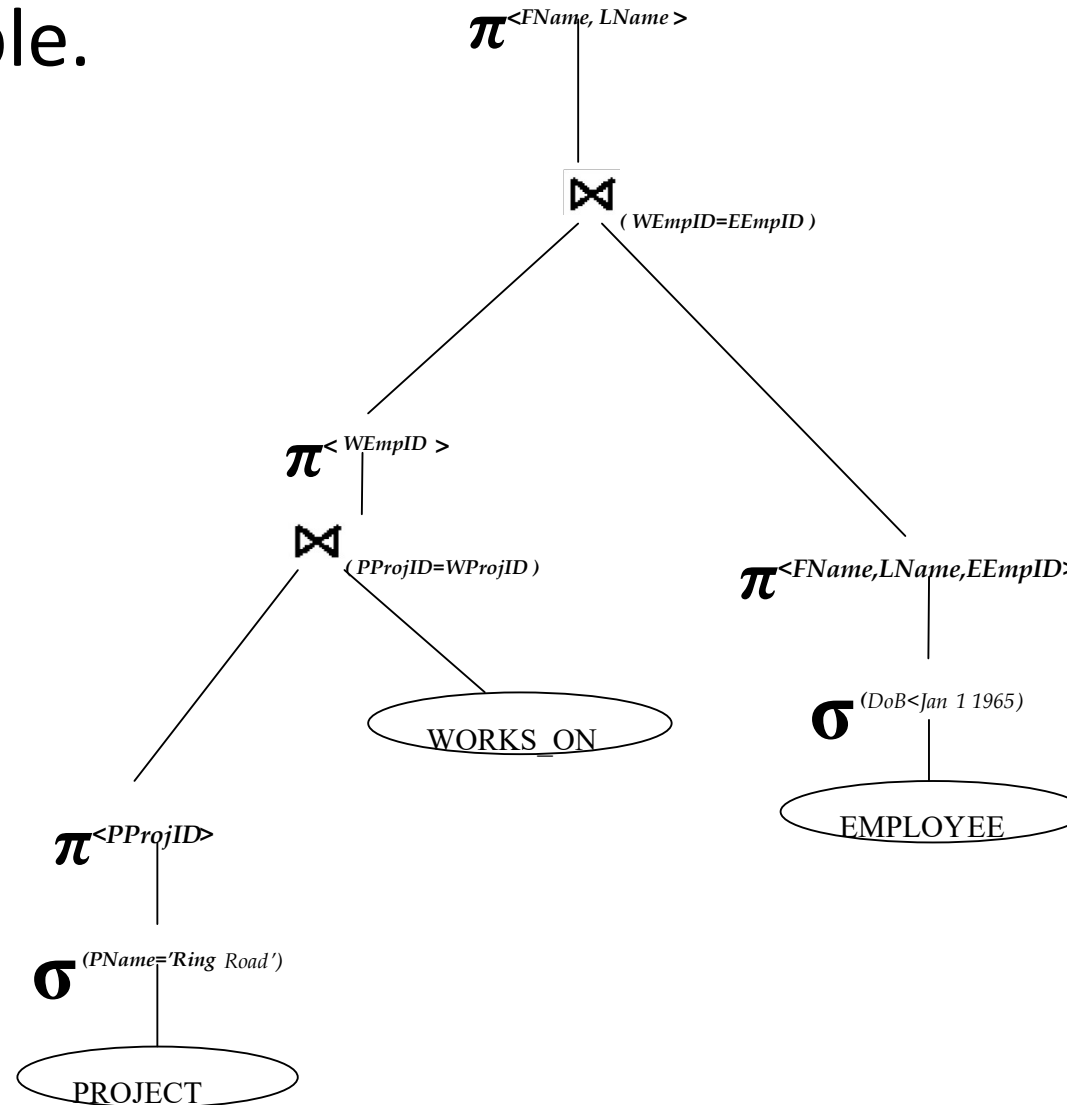
- Using the third step, perform most restrictive operations first.
- From the query given we can see that selection on PROJECT is most restrictive than selection on EMPLOYEE.
- Thus, it is better to perform selection on PROJECT BEFORE on EMPLOYEE. rearrange the nodes to achieve this.



- Using the forth step, Perform Join Operations with the subsequent Selection Operation.



- Using the fifth step, Perform the projection as early as possible.



Cost Estimation Approach to Query Optimization

- The main idea is to minimize the cost of processing a query.
- The cost function is comprised of:
 - I/O cost + CPU processing cost + communication cost + Storage cost
- These components might have different weights in different processing environments.
- The DBMS will use information stored in the system catalogue for the purpose of estimating cost.
- The main target of query optimization is to minimize the size of the intermediate relation.
- The size will have effect in the cost of:
 - Disk Access
 - Data Transpiration
 - Storage space in the Primary Memory
 - Writing on Disk

...CON'T

- The statistics in the system catalogue used for cost estimation purpose are:
 - Cardinality of a relation: the number of tuples contained in a relation currently (r)
 - Degree of a relation: number of attributes of a relation
 - Number of tuples on a relation that can be stored in one block of memory
 - Total number of blocks used by a relation
 - Number of distinct values of an attribute (d)
 - Selection Cardinality of an attribute (S): that is average number of records that will satisfy an equality condition $S=r/d$
- By seeing the above information one could calculate the cost of executing a query and selecting the best strategy, which is with the minimum cost of processing.

Cost Components for Query Optimization

- The costs of query execution can be calculated for the following major process we have during processing.

1) Access Cost of Secondary Storage

- Data is going to be accessed from secondary storage, as a query will be needing some part of the data stored in the database.
- The disk access cost can again be analyzed in terms of:
 - > Searching
 - > Reading, and
 - > Writing, data blocks used to store some portion of a relation.
- The disk access cost will vary depending on the file organization used and the access method implemented for the file organization

...CON'T

2) Storage Cost

- While processing a query, as any query would be composed of many database operations, there could be one or more intermediate results before reaching the final output.
- These intermediate results should be stored in primary memory for further processing.
- The bigger the intermediate relation, the larger the memory requirement, which will have impact on the limited available space.

...CON'T

3) Computation Cost

- Query is composed of many operations.
- The operations could be database operations like reading and writing to a disk, or mathematical and other operations like:
 - > Searching
 - > Sorting
 - > Merging
 - > computation on field values

4) Communication Cost

- In most database systems the database resides in on station and various queries originate from different terminals.
- This will have impact on the performance of the system adding cost for query processing.
- Thus, the cost of transporting data between the database site and the terminal from where the query originate should be analyzed.

Thank You

Questions?

Comments and opinions would be appreciated.